Practical Error Estimation and Step Size Selection

Samuele Paolo Gadaleta and Luca Di Sabatino
Department of Mathematics: Numerical Methods exam

December 2025

1 Introduction

This report presents an enhanced version of the adaptive step method code and algorithm studied during the course for the numerical solution of an initial value problem (IVP). The main objective of this work is to introduce refinements and optimizations aimed at improving the adaptive step size control strategy. Unlike methods based on a constant step size, the adaptive approach dynamically adjusts the step size at each iteration, leading to improved accuracy and computational efficiency.

A central element of the proposed improvements is the accurate formulation and computation of the relative error at each iteration, which plays a fundamental role in determining the appropriate step size. Furthermore, in accordance with the recommendations provided in the referenced paper, an algorithm for the automatic selection of the initial step size has been implemented.

The report begins with a theoretical discussion outlining the modifications introduced to the original code, followed by an analysis of the experimental results. Finally, it concludes with the presentation of the solutions to the exercises proposed in the reference paper.

2 Automatic Step Size Control

Adaptive step-size strategies are commonly adopted in numerical methods to balance computational efficiency with solution accuracy. Instead of advancing the solution using a fixed step size, the method dynamically modifies the step length according to the local behavior of the solution: smaller steps are employed

in regions characterized by rapid variations, while larger steps are allowed when the solution evolves smoothly. The purpose of this approach is to automatically regulate the step size so that a prescribed tolerance on the local error is satisfied.

The local error is estimated by comparing two numerical approximations: y_k, obtained using an explicit Runge–Kutta (ERK) method of order p, and a higher-order approximation $\tilde{y}_k$ of order $q > p$, typically chosen such that $q = p + 1$.
More precisely, given

$$y_k = y_{k-1} + h\,\Phi_p(f, y_{k-1}, t_{k-1}, h),$$

$$\tilde{y}_k = y_{k-1} + h\,\Phi_q(f, y_{k-1}, t_{k-1}, h).$$

The two ERK methods are chosen so that they share the same internal stages K_j, which implies identical coefficients for the matrices A and the vector c in their respective Butcher tableaux. Under this condition, the difference between the two numerical solutions provides a reliable estimate of the local truncation error and can be expressed as:

$$y_k - \tilde{y}_k = \sum_{j=1}^{q} (\tilde{b}_j - b_j) K_j, \qquad b_{p+1} = \cdots = b_q = 0.$$

The numerical solution is advanced using the lower-order approximation, ensuring that the resulting embedded ERK scheme retains order p. Once a tentative step size h has been selected, the algorithm computes the two approximationsy_1 and $\tilde{y}_1$ at the next time level. The error associated with the lower-order solution is then estimated by $|y_1 - \tilde{y}_1|$.
This quantity is required to satisfy the inequity:

$$|y_1 - \tilde{y}_1| \leq sc,$$

$$sc = Atol + max(|y_0|, |y_1|)Rtol,$$

where $Atol$ and $Rtol$ represent user-prescribed absolute and relative tolerances, respectively. When $Atol = 0$, the error control relies solely on relative errors, whereas setting $Rtol = 0$ results in purely absolute error control.
In practical applications, both tolerances are usually nonzero and may vary among different components of the solution vector.

To obtain a scalar error indicator, the following normalized error measure is introduced:

$$err = \left(\frac{1}{n} \sum_{i=1}^{n} \left(\frac{|y_1 - \tilde{y}_1|}{sc} \right)^2 \right)^{1/2}.$$

The value of err is compared against unity to assess whether the current step size is appropriate. Assuming the error behaves asymptotically as

$$err \approx Ch^{p+1},$$

and imposing the condition:

$$1 \approx C h_{\text{opt}}^{p+1},$$

with $p = min(p, q)$, an estimate for the optimal step size is obtained as

$$h_{\text{opt}} = h \left(\frac{1}{\text{err}} \right)^{\frac{1}{p+1}}.$$

For increased numerical stability, a safety factor fac is incorporated into the step-size update. Typical values include 0.8 or 0.9, as well as expressions such as $(0.25)^{\frac{1}{p+1}}$ or $(0.38)^{\frac{1}{p+1}}$. This factor reduces the likelihood that the estimated error exceeds the prescribed tolerance in the subsequent step.

Furthermore, to limit abrupt variations in the step size, the update is constrained by the rule:

$$h_{\text{new}} = h \cdot \min \left(\text{fac}_{\max}, \max \left(\text{fac}_{\min}, \text{fac} \cdot \text{err}^{-1/(q+1)} \right) \right).$$

If $err \leq 1$, the step is accepted and the numerical solution advances using y_1. When the error is significantly smaller than 1 (in particular, $err \leq 0.5$), the step size is increased for the next iteration.
Conversely, if $err > 1$, the step is rejected and recomputed using the reduced step size h_{new}.

An upper bound $fac_{\max}$, typically chosen between 1.5 and 5, is imposed to prevent excessive growth of the step size. While this improves the robustness of the method, overly restrictive values may lead to unnecessary computational cost. In practice, it is often recommended to temporarily set $fac_{\max} = 1$ immediately after a rejected step, thereby avoiding a sudden increase in step size following a failure.

In addition to adaptive step-size control, the choice of the initial step size plays a crucial role in the overall performance of the algorithm. Rather than relying on heuristic considerations, a systematic procedure is employed. Since the local truncation error can be approximated by

$$\text{local error} \approx C h^{p+1} y^{(p+1)}(x_0)$$

and the higher-order derivative $y^{(p+1)}(x_0)$ is generally unavailable, it is replaced by numerical estimates derived from the first and second derivatives of the solution.
The resulting algorithm can be summarized as follows.

1. **Initial evaluation.** Compute $f(x_0, y_0)$ and define

$$d_0 = \|y_0\|, \qquad d_1 = \|f(x_0, y_0)\|.$$

2. **Preliminary step-size estimate.** Set

$$h_0 = 0.01 \, \frac{d_0}{d_1}.$$

If $d_0 < 10^{-5}$ or $d_1 < 10^{-5}$, choose

$$h_0 = 10^{-6}.$$

3. **Explicit Euler trial step.** Compute

$$y_1 = y_0 + h_0 f(x_0, y_0).$$

and evaluate $f(x_0 + h_0, y_1)$.

4. **Second derivative estimate.** Approximate

$$d_2 = \frac{\|f(x_0 + h_0, y_1) - f(x_0, y_0)\|}{h_0}.$$

5. **Refined step-size computation.** Determine h_1 from

$$h_1^{p+1} \max(d_1, d_2) = 0.01.$$

If $\max(d_1, d_2) \leq 10^{-15}$, set

$$h_1 = \max\left(10^{-6}, h_0 \cdot 10^{-3}\right).$$

6. **Final selection.** The initial step size is chosen as

$$h = \min(100h_0, h_1).$$

This procedure generally yields a reasonable initial step size and prevents extremely poor initial choices. As a result, the adaptive ERK method is able to solve initial value problems with improved accuracy and efficiency. The following section presents numerical results obtained by applying this strategy to the same test problem considered in the referenced study, and compares the proposed approach with the ERK method introduced during the lectures and with a standard ERK solver based on a fixed step size.

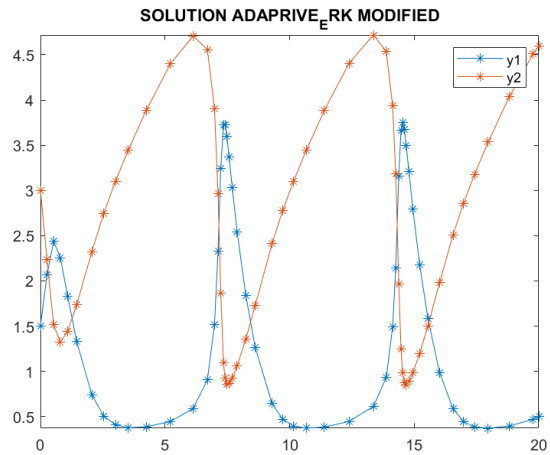
3 Numerical Experiments

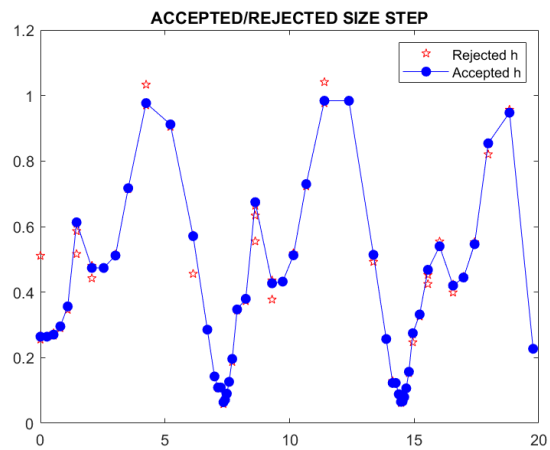
As a representative test case, we consider an initial value problem (IVP) governed by the well-known *Brusselator system*, which is defined by the following set of ordinary differential equations:

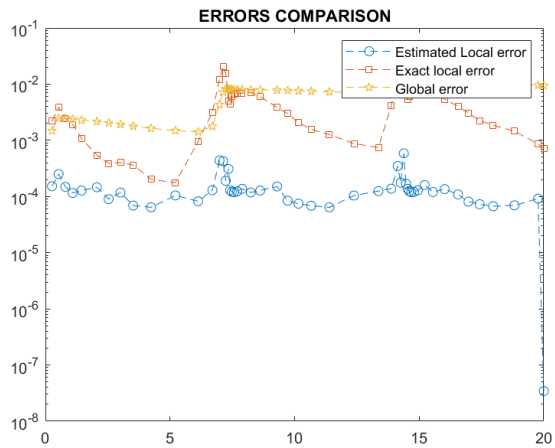
$$\begin{aligned} y_1' &= 1 + y_1^2 y_2 - 4y_1, \\ y_2' &= 3y_1 - y_1^2 y_2. \end{aligned}$$

The system is complemented with the initial conditions $y_1(0) = 1.5$ and $y_2(0) = 3$, and it is integrated over the interval $0 \leq x \leq 20$. Both the absolute and relative tolerances are chosen as $\text{Atol} = \text{Rtol} = 10^{-4}$.

The problem is solved using the improved implementation of the embedded explicit Runge–Kutta (ERK) method developed in this work, which incorporates the adaptive step-size control strategy and refinements discussed in the previous section. In particular, the *Kutta–Fehlberg embedded pair of order* 4(5) is employed. The numerical results obtained from this experiment are reported below.







1. The figure illustrates the qualitative behavior of the numerical solution over the integration interval.

2. The figure compares two different step sizes h. The red curve corresponds to a rejected step size, while the blue curve represents an accepted one. In both cases, the analysis is performed with respect to the temporal variable.

3. The figure shows the evolution, as a function of time, of three different error measures: the exact local error, the estimated local error, and the global error. In addition, a clear correlation can be observed between the smallest values of the step size h and the peaks of the local errors.

4 Numerical comparisons

A comparative analysis is performed among three explicit Runge–Kutta (ERK) methods. The first method adopts a fixed step size, whereas the second employs an adaptive strategy based on a standard step-size control mechanism. The third method further extends the adaptive approach by introducing a dependence on a safety factor, denoted by fac, whose value is selected through a parameter flag taking values 0, 1, 2, or 3. Each value of the flag corresponds to a specific choice of the safety factor.

For all considered methods, including their modified variants, several performance indicators are examined and reported. These include the total number of integration steps, the maximum and minimum step sizes attained during the computation, the relative error—computed by treating the solution produced by a standard ODE solver as the reference solution—and, finally, the total number of function evaluations.

COMPARE THE METHODS

	Num steps	Max step	Min step	Rel_Err	Num fnct Eval
Fixed size ERK	21	1	1	NaN	80
Adaptive ERK ST	56	0.96721	0.056626	0.01027	835
Adaptive ERK M0	52	0.98397	0.064139	0.0094382	17077
Adaptive ERK M1	46	1.2001	0.071869	0.010785	617
Adaptive ERK M2	229	0.27459	0.015061	0.011303	92347
Adaptive ERK M3	135	0.4247	0.025205	0.010764	54277

5 Exercises

The following exercises address both theoretical and practical aspects of explicit Runge–Kutta methods and adaptive step-size strategies.

A. Show that Runge's method can be interpreted as the composition of two explicit Euler steps with step size $h/2$, followed by a Richardson extrapolation.

B. Prove that no four-stage Runge–Kutta method of order four admits an embedded formula of order three.

C. Show that the step-size control strategy is invariant under a rescaling of the independent variable. In particular, demonstrate that equivalent sequences of step sizes are generated when the strategy is applied to the following two initial value problems:

$$y' = f(x, y), \quad y(0) = y_0, \quad y(x_{\text{end}}) = ?,$$

and

$$z' = \sigma\, f(\sigma t, z), \quad z(0) = y_0, \quad z\left(\frac{x_{\text{end}}}{\sigma}\right) = ?,$$

provided that the initial step sizes are chosen as h_0 and h_0/σ, respectively.

6 Solutions

A.

Let us recall Euler's method with a full step size h for the following initial value problem:

$$y'(t) = f(t, y), \qquad y(t_n) = y_n \quad \Longrightarrow \quad y_{n+1} = y_n + hf(t_n, y_n).$$

Now, for a half step size $h/2$, one obtains

$$y_{n+\frac{1}{2}} = y_n + \frac{h}{2} f(t_n, y_n), \tag{a}$$

$$y_{n+1} = y_{n+\frac{1}{2}} + \frac{h}{2} f\left(t_n + \frac{h}{2}, y_{n+\frac{1}{2}}\right), \tag{b}$$

where (a) represents the first step and (b) represents the second one.

By inserting (a) into (b), we obtain

$$y_{n+1} = y_n + \frac{h}{2}\left[f(t_n, y_n) + f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right)\right].$$

We now apply Richardson extrapolation. Recall the extrapolation formula

$$\hat{y}_{n+1} = y_{n+1} + \frac{y_{n+1} - w}{2^p - 1},$$

where w denotes the result obtained by performing one single step of size $2h$.

In order to compute w, we perform a standard Euler step with step size $2\left(\frac{h}{2}\right) = h$, namely

$$w = y_n + hf(t_n, y_n).$$

In this case, the order of the method is $p = 1$, since it coincides with the order of the Euler method. By applying Richardson extrapolation, we obtain the desired result. By applying Richardson extrapolation, one has

$$\hat{y}_{n+1} = y_{n+1} + \frac{y_{n+1} - (y_n + hf(t_n, y_n))}{2^1 - 1} = 2y_{n+1} - y_n - hf(t_n, y_n).$$

Substituting the expression of y_{n+1} obtained from the two half-step Euler method, we get

$$2\left[y_n + \frac{h}{2}\left(f(t_n, y_n) + f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right)\right)\right] - y_n - hf(t_n, y_n).$$

Simplifying the expression yields

$$2y_n + hf(t_n, y_n) + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right) - y_n - hf(t_n, y_n) = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right).$$

This is exactly Runge's formula, which in general reads

$$y_{n+1} = y_n + hf\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}f(t_n, y_n)\right).$$

B.

We now state a useful lemma in order to solve the exercise.

Lemma 1. Let U and V be 3×3 matrices such that

$$UV = \begin{pmatrix} a & b & 0 \\ c & d & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \text{with } \det\begin{pmatrix} a & b \\ c & d \end{pmatrix} \neq 0.$$

Then, either $Ve_3 = 0$ or $U^T e_3 = 0$, where

$$e_3 = (0, 0, 1)^T.$$

Let us consider a 4-stage Runge–Kutta method of order 4 and define

$$d_j = b_j - \hat{b}_j, \qquad \forall j = 1, \dots, 4.$$

Here, the coefficients b_j correspond to the order 4 method, while $\hat{b}_j$ denote the coefficients of the embedded formula. We aim to show that $d_j = 0$, that is, $b_j = \hat{b}_j$ for all $j = 1, \dots, 4$. This will lead to a contradiction, since the coefficients $\hat{b}_j$ belong to the embedded formula, which has a lower order (order 3 in our case), and therefore the two methods may exhibit different orders of convergence. Let us introduce the two matrices:

$$U = \begin{pmatrix} b_2 & b_3 & b_4 \\ b_2c_2 & b_3c_3 & b_4c_4 \\ d_2 & d_3 & d_4 \end{pmatrix}, \qquad V = \begin{pmatrix} c_2 & c_2^2 & \sum_j a_{2j}c_j - \dfrac{c_2^2}{2} \\ c_3 & c_3^2 & \sum_j a_{3j}c_j - \dfrac{c_3^2}{2} \\ c_4 & c_4^2 & \sum_j a_{4j}c_j - \dfrac{c_4^2}{2} \end{pmatrix}$$

By exploiting the relations between the coefficients of the Butcher matrices corresponding to the two methods — relations which guarantee their respective convergence orders of 4 and 3 — we derive the following expression for the product of the matrices U and V:

$$UV = \begin{pmatrix} \frac{1}{2} & \frac{1}{3} & 0 \\ \frac{1}{3} & \frac{1}{4} & 0 \\ 0 & 0 & 0 \end{pmatrix}, \qquad \text{with } \det \begin{pmatrix} \frac{1}{2} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{4} \end{pmatrix} \neq 0.$$

The conditions that ensure order 3 convergence, and therefore, when combined with other conditions, also order 4, are the following:

$$\sum_i b_i = b_1 + b_2 + b_3 + b_4 = 1,$$

$$\sum_i b_i c_i = b_2c_2 + b_3c_3 + b_4c_4 = \frac{1}{2},$$

$$\sum_i b_i c_i^2 = b_2c_2^2 + b_3c_3^2 + b_4c_4^2 = \frac{1}{3},$$

$$\sum_{i=2}^{4} \sum_{j=1}^{i-1} b_i a_{ij} c_j = b_3a_{32}c_2 + b_4\big(a_{42}c_2 + a_{43}c_3\big) = \frac{1}{6}.$$

According to the previous lemma, either $Ve_3 = 0$ or $U^T e_3 = 0$. However, Ve_3 coincides with the last column of the matrix V, and this implies

$$\begin{pmatrix} \sum_j a_{2j}c_j - c_2^2/2 \\ \sum_j a_{3j}c_j - c_3^2/2 \\ \sum_j a_{4j}c_j - c_4^2/2 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}.$$

Since $c_1 = 0$, this implies that

$$\sum_j a_{ij} c_j - \frac{c_i^2}{2} = -\frac{c_i^2}{2} = 0, \forall i = 2, 3, 4.$$

This leads to a contradiction. So $U^T e_3$ has necessarily to be equal to 0, and we obtain:

$$d_2 = d_3 = d_4 = 0, \qquad b_j = \hat{b}_j, \qquad \forall j = 2, 3, 4.$$

We may apply the following condition:

$$\sum_{j=1}^{4} b_j = \sum_{j=1}^{4} \hat{b}_j,$$

$$\sum_{j=1}^{4} b_j - \sum_{j=1}^{4} \hat{b}_j = 0,$$

$$\sum_{j=1}^{4} \left(b_j - \hat{b}_j \right) = 1 - 1 = 0.$$

On the other hand,

$$\sum_{j=1}^{4} \left(b_j - \hat{b}_j \right) = \sum_{j=1}^{4} d_j = 0,$$

which implies $d_1 = 0$ and therefore

$$b_j = \hat{b}_j, \qquad \forall j = 1, \ldots, 4.$$

This shows that no 4-stage Runge–Kutta method of order 4 admits an embedded formula of order 3.

C.

The adaptive step-size strategy determines a variable step size h at each time interval, based on the evaluation of the relative error associated with the current approximation. This error is estimated by comparing the results obtained by distinct numerical methods. If the calculated error satisfies the condition err $\leq$ 1, the step size h is maintained. Otherwise, it is recalculated according to the following rule:

$$h = h \cdot \left(\frac{1}{\text{err}} \right)^{\frac{1}{q+1}}.$$

Consequently, the step sizes h are determined by the relative error computed at each iteration. In the context of our problem, if the relative error is identical

for both the equations y and z, then the corresponding step sizes h are identical as well. Specifically, for the two equations considered:

$$\mathrm{err}_1 = \frac{\|y_k - \hat{y}_k\|}{C_1}, \qquad \mathrm{err}_2 = \frac{\|z_k - \hat{z}_k\|}{C_2},$$

where C_1 and C_2 are constants that depend on the initial conditions. However, since $y(0) = y_0 = z(0)$, it is sufficient to focus solely on the numerators. Therefore, we obtain

$$\|y_k - \hat{y}_k\| \sim h^{p+1} y^{(p+1)}(0), \qquad \|z_k - \hat{z}_k\| \sim \left(\frac{h}{\sigma}\right)^{p+1} z^{(p+1)}(0),$$

where p denotes the order of convergence of the embedded Runge–Kutta method. The term h/σ appears in the second equation because, in the case of the y-problem, the initial step size is h_0, whereas for the z-problem, the initial step size is h_0/σ. Moreover, by the definition of the z-problem, we have

$$z^{(p+1)}(0) = \sigma^{p+1} y^{(p+1)}(0).$$

Therefore, we obtain the following:

$$\|z_k - \hat{z}_k\| \sim \left(\frac{h}{\sigma}\right)^{p+1} \sigma^{p+1} y^{(p+1)}(0) = h^{p+1} y^{(p+1)}(0).$$

Thus, the local error remains identical at each time step for both problems. As a consequence, the sequences of step sizes h generated by the adaptive strategy will also coincide.

This property no longer holds if the error estimate err is replaced by err$/h$ and q by $q-1$, corresponding to the so-called *error per unit step*.

In fact, if we consider

$$\mathrm{err}_1^{\mathrm{norm}} = \frac{\mathrm{err}_1}{h} \sim h^q \left| y^{(q+1)}(0) \right|, \tag{*}$$

$$\mathrm{err}_2^{\mathrm{norm}} = \frac{\mathrm{err}_2}{h} \sim \left(\frac{h}{\sigma}\right)^q \sigma^{q+1} \left| y^{(q+1)}(0) \right| = h^q \sigma \left| y^{(q+1)}(0) \right|, \tag{**}$$

we can observe that the errors exhibit a different behavior, due to the presence of the factor σ, which persists in the error expression of the rescaled problem.